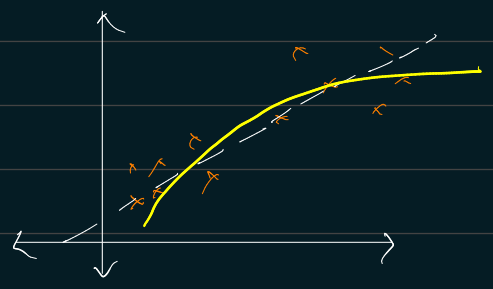


$\theta_0 + \theta_1 x$ → linear

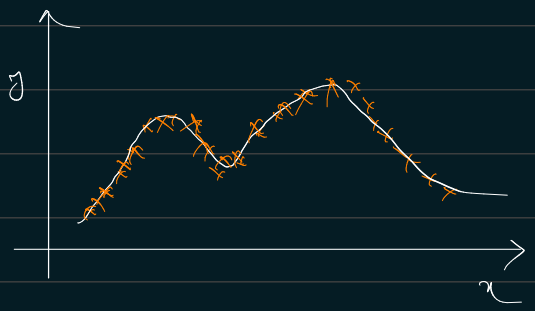
$\theta_0 + \theta_1 x + \theta_2 x^2$ → quadratic → curve returns back

$\theta_0 + \theta_1 x + \theta_2 \sqrt{x}$

$\theta_0 + \theta_1 x + \theta_2 \log x$

→ Feature selection is choosing which feature which represents x (x^2, $\log x$, $\sqrt{x}$ ---) that best fits the data.

✱ Locally weighted regression :-



ML algorithms :-

1- Parametric learning algorithms :-

Fit fixed set of parameters (θ_i) to data.

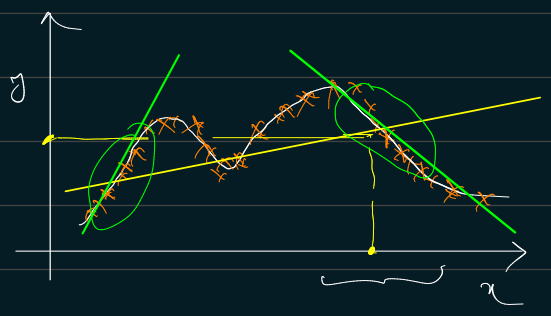
Ex- Linear regression.

2- Non Parametric learning algorithm :-

The amount of data/ parameters you need to keep grows (linearly) with the size of data.

Ex- Locally weighted regression.

→ Not suitable for large data as it should be kept in mem to make predictions.



To evaluate h at a certain value of input, x :

Linear Regression: Fit θ to minimize the cost function

$$\frac{1}{2m} \sum_{i=0}^{m} \left(\theta^T x^{(i)} - y^{(i)} \right)^2$$

Return $\theta^T x$

Locally weighted Regression: Fit θ to minimize a modified cost function

$$\sum_{i=0}^{m} w^{(i)} \left(\theta^T x^{(i)} - y^{(i)} \right)^2$$

where $w^{(i)}$ is a weighting function

$$w^{(i)} = \exp\left(- \frac{(x^{(i)} - x)^2}{2\tau^2} \right)$$

$\Rightarrow$ If $|x^{(i)} - x|$ is small, then

$$w^{(i)} \approx e^{-0} \approx 1$$

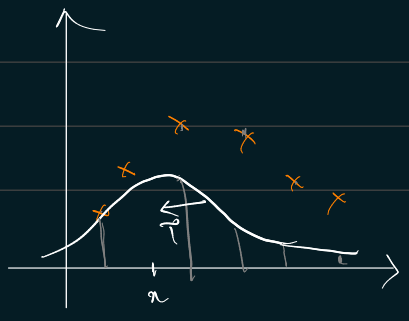
$\Rightarrow$ If $|x^{(i)} - x|$ is large, then

$$w^{(i)} \approx 0$$

$$\boxed{\sum_{i=0}^{m} w^{(i)} \left(\theta^T x^{(i)} - y^{(i)} \right)^2}$$

→ If example, $x^{(i)}$ is far from where we want to make prediction, then multiple the error term by ~ 0

→ If it is close, multiple it with ~ 1.



τ = bandwidth paramter

$\tau \longleftrightarrow$ overfitting

$\tau \leftrightarrow$ underfitting

→ Not a good algo to extrapolate the prediction

✱ Why least Squares (not ^3 / ^4 ---) ?

For housing price prediction,

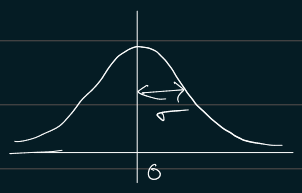
Assume $y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$ → IID (independently & identically distributed)

$\varepsilon^{(i)}$ ↓ error term (includes unmodelled effects & random noise)

$\varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$

↓ is distributed; ↓ Normal dist with mean = 0 & sd = σ^2

$$p(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\,\sigma} e^{\left(\frac{-(\varepsilon^{(i)})^2}{2\sigma^2}\right)}$$



⇒ This implies

$$p(y^{(i)} \mid x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

(mean; variance; "parameterized by")

$$y^{(i)} \mid x^{(i)}; \theta \sim \mathcal{N}(\theta^T x^{(i)}, \sigma^2)$$

Read as, "The random variable y given x parameterized by θ is distributed Gaussian with mean $\theta^T x^{(i)}$ & variance σ^2"

$$L(\theta) = p(\vec{y} \mid x; \theta)$$

Likelihood of parameter, θ $= \prod_{i=1}^{m} p(y^{(i)} \mid x^{(i)}; \theta)$

$$= \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Log likelihood

$$l(\theta) = \log L(\theta) = \log \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

$$= \sum_{i=1}^{m} \left[\log \frac{1}{\sqrt{2\pi}\sigma} + \log \exp\left(\quad \right) \right]$$

$$= m \cdot \log \frac{1}{\sqrt{2\pi}\sigma} + \sum_{i=1}^{m} \frac{-\left(y^{(i)} - \theta^T x^{(i)}\right)^2}{2\sigma^2}$$

Maximum likelihood estimation (mle) → one of the methods in statistics for estimating parameters.

Choose θ to maximize $\ell(\theta)$ (mathematically easier to work on $l(\theta)$)

$$l(\theta) = \underbrace{m \cdot \log \frac{1}{\sqrt{2\pi}\sigma}}_{\text{Constant}} + \sum_{i=1}^{m} \frac{-\left(y^{(i)} - \theta^T x^{(i)}\right)^2}{2\sigma^2}$$

↳ Constant

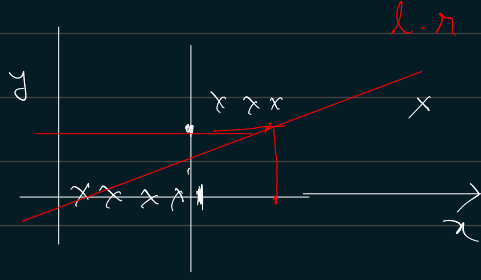
$\Rightarrow$ Choose θ to minimize $\frac{1}{2} \sum_{i=1}^{m} \left(y^{(i)} - \theta^T x^{(i)}\right)^2 = J(\theta)$

(Cost function)

$\Rightarrow$ If error terms, $\varepsilon^{(i)}$ are Gaussian and IID & want to use maximum likelihood estimation then use least sq. errors.

Classification :-

$y \in \{0, 1\}$ (binary classification)

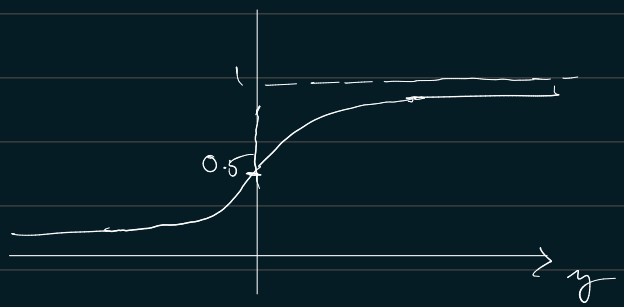


Logistic Regression :-

Want $h_\theta(x) \in [0, 1]$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \in [0, 1]$$

$$g(z) = \frac{1}{1 + e^{-z}} \rightarrow \text{'Sigmoid' or 'Logistic' function.}$$



$$P(y=1 \mid x; \theta) = h_\theta(x)$$

prob. of tumor is malignant ($y=1$) given size of tumor parametrized by θ is equal to the output of hypothesis.

$$P(y=0 \mid x; \theta) = 1 - h_\theta(x) \qquad y \in \{0, 1\}$$

Combining both the equations :-

$$P(y \mid x; \theta) = h_\theta(x)^y \, (1 - h_\theta(x))^{1-y}$$

If $y=1 \Rightarrow (1-h_\theta(x))^{1-1} = 0$

$P(y=1 \mid x;\theta) = h_\theta(x)$

If $y=0 \Rightarrow h_\theta(x)^0 = 1$

$P(y=0 \mid x;\theta) = 1-h_\theta(x)$

$$L(\theta) = P(\vec{y} \mid x;\theta)$$
$$= \prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)};\theta)$$
$$= \prod_{i=1}^{m} h_\theta(x^{(i)})^{y^{(i)}} \cdot \left(1-h_\theta(x^{(i)})\right)^{1-y^{(i)}}$$

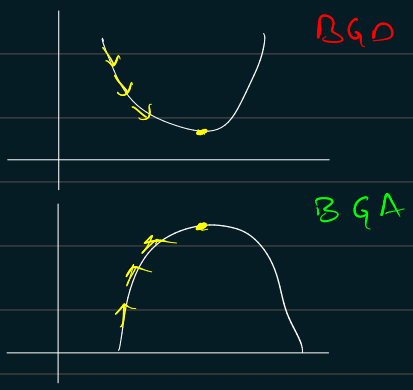
log likelihood,

$$l(\theta) = \log L(\theta) = \sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log\left(1-h_\theta(x^{(i)})\right)$$

Choose θ to maximize $l(\theta)$ using

Batch gradient ascent :-

$$\theta_j := \theta_j + \alpha \frac{\partial}{\partial \theta_j} l(\theta)$$

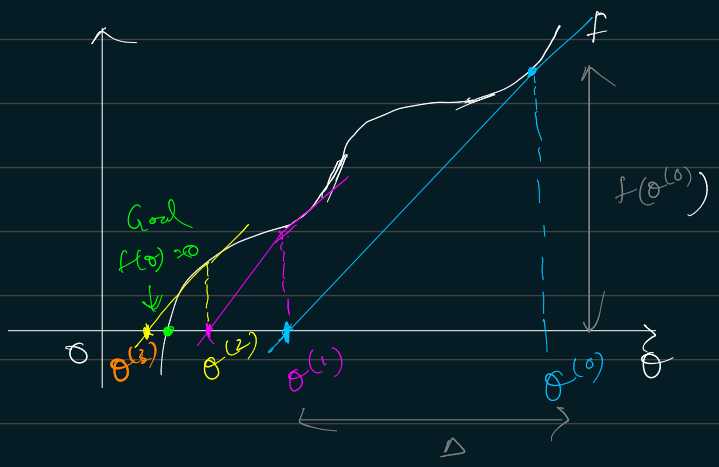


$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \underbrace{\left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}}_{\frac{\partial}{\partial \theta_j} l(\theta)}$$

Newton's method :-

Suppose we have a function, f & want to find $f(\theta)$ such that $f(\theta) = 0$

[want to maximize $l(\theta)$ i.e $l'(\theta) = 0$].



$$\theta^{(1)} = \theta^{(0)} - \Delta$$

$$f'(\theta^{(0)}) = \frac{f(\theta^{(0)})}{\Delta}$$

$$\Delta = \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$$

$$\Rightarrow \theta^{(t+1)} = \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})}$$

Let $f(\theta) = l'(\theta)$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{l'(\theta^{(t)})}{l''(\theta^{(t)})}$$

Quadratic Convergence :-

$0.01 \longrightarrow 0.0001 \longrightarrow 0.00000001$ error

1st 2nd 3rd iteration

The function smoothens such that the no. of least significant digits

of error doubles at every iteration.

→ Therefore Newton's method converges rapidly compared to gradient ascent.

When θ is a vector. $(\theta \in \mathbb{R}^{n+1})$

$$\theta^{(t+1)} := \theta^{(t)} + H^{-1} \nabla_\theta l$$

$\nabla_\theta l$ → vector of derivatives $\mathbb{R}^{n+1}$

H → Hessian matrix $\in \mathbb{R}^{n+1 \times n+1}$

$$H_{ij} = \frac{\partial^2 l}{\partial \theta_i \partial \theta_j}$$

→ So, Newton's method is much more expensive (Computing inverse matrix (H^{-1}) for high no. of parameters, pd of more parameters)

→ Good for 10 ~ 15 parameters - Converges within 50 - 100 iterations.